

AI 에이전트

전문가 양성 과정

데이터 구조



Contents

1. 컨테이너

- 리스트
- 튜플
- 딕셔너리
- 세트

2. 형 변환

3. 참고 자료

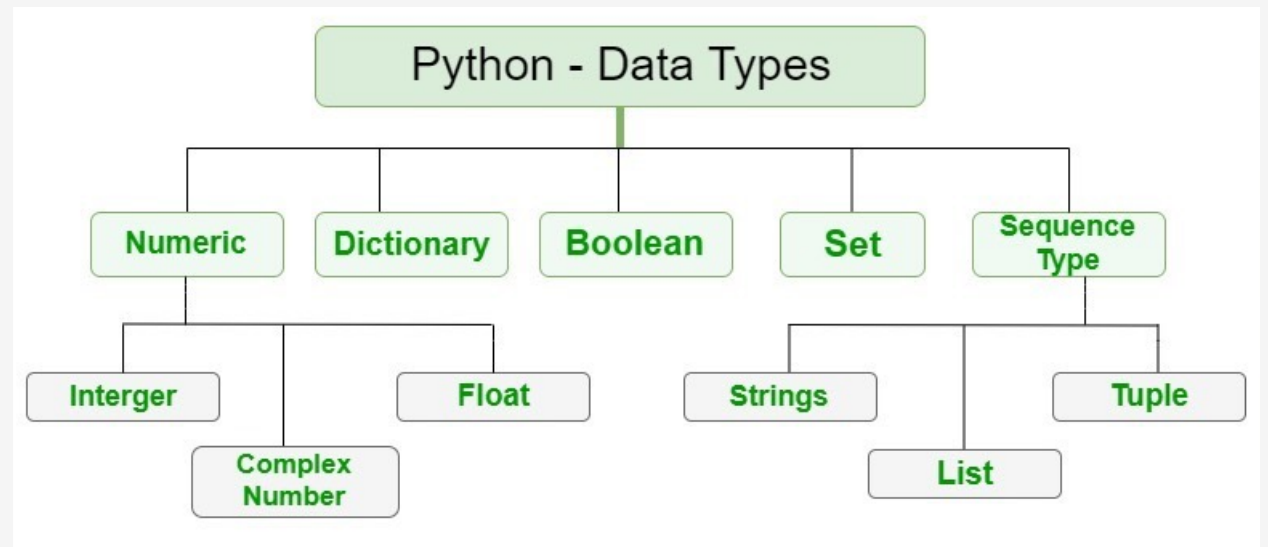
- 유니코드
- 복사

제 1장. 컨테이너

1-1 리스트

리스트

- 컨테이너 : 여러 개의 데이터를 담을 수 있는 것
 - 데이터를 담는 상자
 - 다른 자료형을 저장할 수 있음
- 컨테이너 분류
 - 시퀀스형 : 순서가 있는 데이터
 - 비시퀀스형 : 순서가 없는 데이터



리스트

- 여러 값을 순서대로 담는 자료형
 - 대괄호([])사용
 - 값과 값은 쉼표 (,)로 구분
 - 다양한 값과 다양한 자료형이 섞인 리스트도 가능

```
# 리스트 변수명 = [foo, bar, baz]
numbers = [10, 20, 30, 40]
names = ["철수", "영희", "민수"]
```

```
numbers = [1, 2, 3]
names = ["Jay", "민수", "영희"]
mixed = ["Jay", 30, True]
```

리스트

- 여러 값을 순서대로 담는 자료형
 - 변수를 따로 관리하면 관리가 불편
 - 관련 있는 데이터를 하나로 관리할 수 있어 코드가 깔끔
 - 반복문과 함께 사용하기 좋음

```
score1 = 90  
score2 = 80  
score3 = 70
```



```
scores = [90, 80, 70]
```

리스트

- 여러 값을 순서대로 담는 자료형
 - 리스트는 값을 순서대로 저장
 - 각 값에는 번호(인덱스, index)가 붙는다.
 - Python의 인덱스는 0부터 시작, 뒤부터 셀 땐 -1부터 시작

```
fruits = ["apple", "banana", "orange"]
```

```
print(fruits[0])  
print(fruits[2])  
print(fruits[-1])  
print(fruits[-2])
```


리스트

- 여러 값을 순서대로 담는 자료형
 - 슬라이싱 : 리스트의 일부만 잘라내는 문법
 - `list[start:end]` : start부터 시작해서 end 전까지 가져옴

```
numbers = [10, 20, 30, 40, 50]
```

```
print(numbers[0:3])
```

```
print(numbers[1:4])
```

```
print(numbers[:3])
```

```
print(numbers[2:])
```

리스트

- 여러 값을 순서대로 담는 자료형
 - 리스트 안에 있는 값을 바꿀 수 있다. (인덱스 사용)
 - 리스트는 수정 가능한 자료형

```
fruits = ["apple", "banana", "orange"]  
  
fruits[1] = "grape"  
  
print(fruits)
```

리스트

- 여러 값을 순서대로 담는 자료형
 - 리스트 연산
- + 를 사용해 두 리스트를 하나로 합칠 수 있고, * 를 사용해 반복할 수 있음

```
a = [1, 2, 3]
```

```
b = [4, 5, 6]
```

```
c = a + b
```

```
print(c) # [1, 2, 3, 4, 5, 6]
```

```
print(a * 3) # [1, 2, 3, 1, 2, 3, 1, 2, 3]
```

리스트

- 여러 값을 순서대로 담는 자료형
 - 리스트 안에 있는 값을 `append()`를 사용해 추가할 수 있다
 - 값을 삭제할 때는 `remove()` 사용

```
fruits = ["apple", "banana"]  
  
fruits.append("orange")  
  
print(fruits)
```

```
fruits = ["apple", "banana", "orange"]  
  
fruits.remove("banana")  
  
print(fruits)
```

리스트

- 여러 값을 순서대로 담는 자료형
 - 인덱스를 기준으로 삭제할 때는 pop() 사용
 - 꺼낸 값을 변수에 저장할 수 있음

```
fruits = ["apple", "banana", "orange"]  
  
popped_data = fruits.pop(1)  
  
print(fruits)  
print(popped_data)
```

리스트

- 여러 값을 순서대로 담는 자료형
 - 리스트의 길이는 len()으로 확인 가능
 - 데이터가 몇 개인지 확인할 때 사용
 - 검색 결과, 메시지 개수 확인 시 사용

```
fruits = ["apple", "banana", "orange"]  
print(len(fruits))
```

리스트

- 여러 값을 순서대로 담는 자료형
 - 리스트 안에 또 다른 컨테이너 (리스트, 딕셔너리 등)가 들어갈 수 있다.

```
boxes = ['box1', 'box2', ['apple', 'banana'], 'box3']  
  
print(boxes[0])  
print(boxes[2])  
print(boxes[2][0])  
print(boxes[2][1])
```

- 'Life' 문자열만 꺼내려면?

```
list_data = [1, 2, ['a', 'b', ['Life', 'is']]]
```

리스트

- 여러 값을 순서대로 담는 자료형
 - `list()`는 다른 데이터를 리스트로 바꿀 때 사용
 - 문자열을 넣으면 글자 하나하나가 리스트 요소로 변환

```
text = "Python"

letters = list(text)

print(letters)
print(type(letters)) # <class 'list'>
```


리스트

- 여러 값을 순서대로 담는 자료형
 - 자주 사용하는 리스트 함수

함수	설명	예시
<code>append()</code>	맨 뒤에 값 추가	<code>items.append("마우스")</code>
<code>insert()</code>	원하는 위치에 값 추가	<code>items.insert(1, "마우스")</code>
<code>remove()</code>	특정 값을 삭제	<code>items.remove("마우스")</code>
<code>pop()</code>	특정 위치의 값을 꺼내고 삭제	<code>items.pop(1)</code>
<code>sort()</code>	리스트 정렬	<code>scores.sort()</code>
<code>reverse()</code>	순서 뒤집기	<code>items.reverse()</code>
<code>count()</code>	특정 값 개수 세기	<code>items.count("사과")</code>
<code>index()</code>	특정 값의 위치 찾기	<code>items.index("사과")</code>
<code>len()</code>	리스트 길이 확인	<code>len(items)</code>

1-2 튜플

튜플

- 여러 값을 순서대로 담는 자료형
 - 리스트와 유사하지만, 튜플은 한 번 만들면 요소 값을 수정할 수 없다.
→ 바뀌면 안 되는 데이터에 사용
- 소괄호 () 사용
- 값과 값은 쉼표 (,)로 구분

```
location = (37.5665, 126.9780)
names = ("철수", "영희", "민수")

print(location)
print(names)
print(type(location)) # <class 'tuple'>
```

튜플

- 여러 값을 순서대로 담는 자료형
 - 리스트와 유사하지만, 튜플은 한 번 만들면 요소 값을 수정할 수 없다.

```
names = ("철수", "영희", "민수")
```

```
names[0] = "영수"
```

```
"""
```

```
TypeError: 'tuple' object does not support item assignment
```

```
"""
```

튜플

- 여러 값을 순서대로 담는 자료형
 - 시퀀스형이기 때문에 인덱싱 및 슬라이싱 가능

```
numbers = (10, 20, 30, 40, 50)
```

```
print(numbers[0])  
print(numbers[1])  
print(numbers[0:3])  
print(numbers[:3])  
print(numbers[2:])
```

튜플

- 여러 값을 순서대로 담는 자료형
 - tuple()는 다른 데이터를 튜플로 바꿀 때 사용
 - 문자열을 넣으면 글자 하나하나가 튜플 요소로 변환

```
text = "Python"

letters = tuple(text)

print(letters)
print(type(letters)) # <class 'tuple'>
```

튜플

- 여러 값을 순서대로 담는 자료형
 - 언패킹 : 튜플 안의 값을 여러 변수에 나누어 담을 수 있다
 - 변수와 요소의 수가 같아야 함
 - 변수가 부족한 경우 : `ValueError: too many values to unpack` 에러 발생
 - 변수가 많은 경우 : `ValueError: not enough values to unpack` 에러 발생

```
user = ("Jay", True, "Student")

name, married, job = user

print(name)
print(married)
print(job)
```

튜플

- 여러 값을 순서대로 담는 자료형
 - 언패킹 : 튜플 안의 값을 여러 변수에 나누어 담을 수 있다
 - 해결 방법
 - 필요 없는 값은 _ 로 받기, 남는 값은 * 로 한 번에 받기

```
user = ("Jay", True, "Student")

name, married, _ = user

print(name)
print(married)
```

```
user = ("Jay", True, "Student", "Seoul")

name, married, *others = user

print(name)
print(married)
print(others)
```


1-3 덕셔너리

딕셔너리

- 키-값 (key-value) 쌍으로 이루어진 자료형
 - 순서가 아닌, 이름표를 붙여 값을 저장
 - 중괄호 {} 사용
 - key : value 형태로 데이터 저장

key	value
name	Jay
married	True
job	Student

```
user = {  
    "name": "Jay",  
    "married": True,  
    "job": "Student"  
}
```

딕셔너리

- 키-값 (key-value) 쌍으로 이루어진 자료형
 - 리스트 VS 딕셔너리
 - 리스트 : 순서가 있어, 순서에 따라 데이터 조회 (인덱싱)
 - 딕셔너리 : 이름표(key)가 있어, 키를 사용해 데이터 조회

```
user = ["Jay", True, "Student"]  
  
print(user[0])  
print(user[1])
```

```
user = {  
    "name": "Jay",  
    "married": True,  
    "job": " Student "  
}  
  
print(user["name"])  
print(user["married"])
```

딕셔너리

- 키-값 (key-value) 쌍으로 이루어진 자료형
 - 존재하지 않는 key에 접근 → KeyError 발생

```
user = {  
    "name": "Jay",  
    "married": True,  
    "age": 30,  
    "job": "Student"  
}  
  
print(user["language"]) # KeyError: 'language'
```

딕셔너리

- `get()`으로 안전하게 꺼내기
 - `key`가 없으면 오류를 내지 않고 `None`을 반환

```
print(user.get("language", "Python")) # Python  
print(user.get("language"))           # None
```

딕셔너리

- 키-값 (key-value) 쌍으로 이루어진 자료형
 - 딕셔너리에 값 추가, 수정, 삭제
 - 새로운 key에 값을 넣으면 딕셔너리에 항목이 추가
 - 이미 존재하는 key에 새 값을 넣으면 value가 수정
 - pop()을 사용해 데이터 삭제

```
user = {  
    "name": "Jay",  
    "married": True,  
    "age": 30,  
    "job": "Student"  
}  
  
user["language"] = "Python"  
user["age"] = 31  
is_married = user.pop("married")  
  
print(user)  
print(is_married)
```

딕셔너리

- 키-값 (key-value) 쌍으로 이루어진 자료형
 - key가 있는지 확인 (in)
 - key는 고유한 값.
 - 중복 시 최종 값을 제외한 나머지는 제외

```
user = {  
    "name": "Jay",  
    "age": 30  
}  
  
print("name" in user)    # True  
print("email" in user)   # False
```

```
user = {  
    "name": "Jay",  
    "age": 30,  
    "age" : 31  
}  
  
print(user)
```

딕셔너리

- 키-값 (key-value) 쌍으로 이루어진 자료형
 - `keys()` : key 목록 확인
 - `values()` : value 목록 확인
 - `items()` : key와 value 쌍 확인

```
user = {  
    "name": "Jay",  
    "age": 30,  
}  
  
print(user.keys())    # dict_keys(['name', 'age'])  
print(user.values())  # dict_values(['Jay', 30])  
print(user.items())   # dict_items([('name', 'Jay'), ('age', 30)])  
  
for key in user.keys():  
    print(key) # name, age
```


딕셔너리

- 키-값 (key-value) 쌍으로 이루어진 자료형
 - 중첩 딕셔너리 : 딕셔너리 안에 또 다른 딕셔너리가 들어갈 수 있다.

```
user = {  
    "name": "Jay",  
    "profile": {  
        "age": 30,  
        "job": "Student"  
    },  
    "languages": ["Python", "JavaScript"]  
}
```

```
print(user["profile"]["age"]) # 30  
print(user["profile"]["job"]) # Student  
print(user["languages"][0]) # Python
```

딕셔너리

- 딕셔너리 : 키-값 (key-value) 쌍으로 이루어진 자료형
- 자주 쓰는 딕셔너리 함수

함수	설명	예시
<code>get()</code>	key로 값 가져오기	<code>user.get("name")</code>
<code>keys()</code>	key 목록 확인	<code>user.keys()</code>
<code>values()</code>	value 목록 확인	<code>user.values()</code>
<code>items()</code>	key-value 쌍 확인	<code>user.items()</code>
<code>pop()</code>	값을 꺼내고 삭제	<code>user.pop("age")</code>
<code>update()</code>	여러 값 추가/수정	<code>user.update({"age": 31})</code>
<code>clear()</code>	모든 값 삭제	<code>user.clear()</code>
<code>len()</code>	항목 개수 확인	<code>len(user)</code>

1-4 세트

세트

- 중복을 허용하지 않는 컨테이너
 - 여러 값을 담을 수 있지만, 같은 값은 한 번만 저장 (수학의 집합과 유사)
 - 중괄호 {} 사용

```
list_set = [1, 2, 3, 3, 4]
str_data = "bannana"

print(set(list_set)) # {1, 2, 3, 4}
print(set(str_data)) # {'b', 'a', 'n'} - 순서가 보장되지 않음
```

👉 중복을 허용하지 않음, 순서가 없음

세트

- 중복을 허용하지 않는 컨테이너
 - `set()`으로 중복 제거 후 다시 리스트로 바꾸기
 - 단, `set`은 순서가 없음
 - 원래 순서가 유지되지 않을 수 있어, 중복 제거가 목적일 때 사용

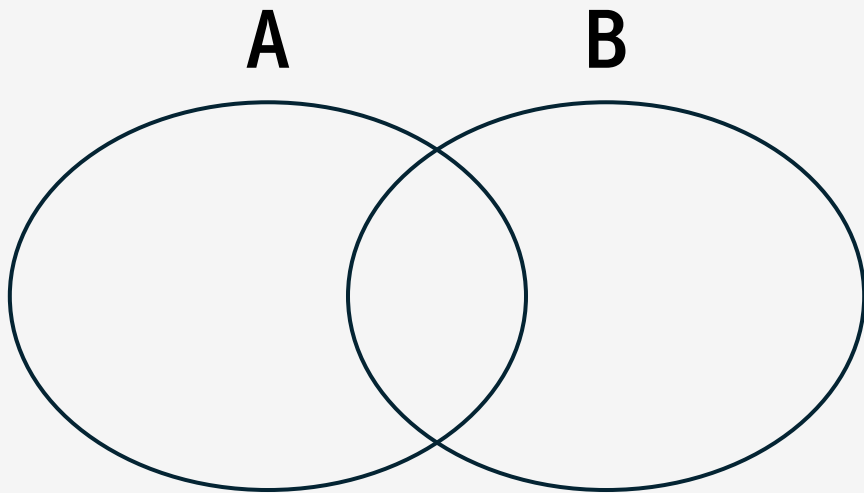
```
numbers = [3, 1, 2, 2, 3, 1]

unique_numbers = list(set(numbers))

print(unique_numbers) # [1, 2, 3]
```

세트

- 중복을 허용하지 않는 컨테이너
 - 집합 연산



연산	의미	기호
합집합	두 집합의 모든 값	
교집합	두 집합에 모두 있는 값	&
차집합	앞 집합에만 있는 값	-
대칭 차집합	한쪽에만 있는 값	^

세트

- 중복을 허용하지 않는 컨테이너
 - 집합 연산

```
python_students = {1,2,3,4,5}
ai_students = {3,4,5,6,7}

print(python_students | ai_students) # {1, 2, 3, 4, 5, 6, 7}
print(python_students & ai_students) # {3, 4, 5}
print(python_students - ai_students) # {1, 2}
print(ai_students - python_students) # {6, 7}
print(python_students ^ ai_students) # {1, 2, 6, 7}
```

세트

- 중복을 허용하지 않는 컨테이너
 - set()은 중복된 값을 제거
 - G와 g는 서로 다른 문자로 판단 → 둘 다 남음
 - Python은 Unicode 기반 (<https://docs.python.org/ko/3/howto/unicode.html>)

```
str_data = "Gangnam"

print(set(str_data)) # {'a', 'G', 'g', 'n', 'm'}

print("G" == "g") # False
```


제 2장. 형 변환

2-1 형 변환

형 변환

- 파이썬에서 데이터 형태 (type)은 변환할 수 있음
 - 숫자, 문자열, 리스트, 튜플, 집합 등 다양한 자료형
 - 상황에 따라 자료형을 바꿔야 계산하거나 출력 가능

```
number = "10"
```

```
print(type(number)) # <class 'str'>
```

```
a = "10"
```

```
b = "20"
```

```
print(a + b) # 1020
```

형 변환

- 파이썬에서 데이터 형태 (type)은 변환할 수 있음
 - int()
 - str()
 - f-string을 더 많이 사용

```
a = "10"
b = "20"

print(a + b) # 1020

print(int(a) + int(b)) # 30
```

```
age = 30

message = "나이는 " + age + "살입니다."

print(message)
# TypeError: can only concatenate str (not "int") to str
```

```
message = "나이는 " + str(age) + "살입니다."

print(message)
print(f"나이는 {age}살입니다.")
```

형 변환

- 파이썬에서 데이터 형태 (type)은 변환할 수 있음
 - `input()` : 프로그램 실행 중 사용자에게 값을 입력 받을 때 사용
 - 입력 받은 값은 항상 문자열

```
age = input("나이를 입력하세요: ")

print(age)          # 30
print(type(age))    # <class 'str'>
print(age + 1)      # TypeError: can only concatenate str (not "int") to str
print(int(age) + 1) # 31
```

형 변환

- `bool()`
 - Python에서 False로 보는 값
 - `False`
 - `0`
 - `0.0`
 - `""`
 - `[]`
 - `()`
 - `{}`
 - `set()`
 - `None`

```
print(bool(1)) # True
print(bool(0)) # False
print(bool("hello")) # True
print(bool("")) # False
print(bool([])) # False
print(bool(None)) # False
```

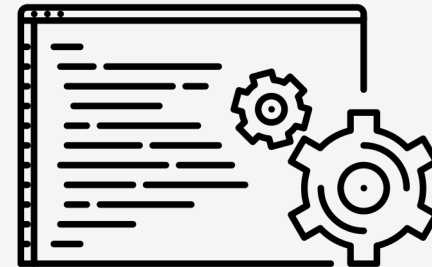
제 3장.

참고 자료

3-1 유니코드

유니코드

- 컴퓨터는 문자를 어떻게 이해할까?
 - 컴퓨터는 문자를 그대로 이해할 수 없음
 - 사람에게는 문자, 컴퓨터에선 모든 데이터가 숫자
 - 문자마다 숫자 번호를 붙이는 약속이 필요



번역기

01101110101
10111101001
11000111101

유니코드

- ASCII
 - 영어 알파벳, 숫자, 일부 특수 문자 등 128개의 문자를 표현하던 오래된 문자 체계
 - 표현할 수 있는 문자가 적음 (한글, 이모지, 다양한 언어를 표현하기 어려움)
- Unicode
 - 전 세계 문자를 담는 더 큰 체계 수립
 - Unicode는 ASCII 문자를 포함

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
0_	U+0000 NUL	U+0001 SOH	U+0002 STX	U+0003 ETX	U+0004 EOT	U+0005 ENQ	U+0006 ACK	U+0007 BEL	U+0008 BS	U+0009 HT	U+000A LF	U+000B VT	U+000C FF	U+000D CR	U+000E SO	U+000F SI
1_	U+0010 DLE	U+0011 DC1	U+0012 DC2	U+0013 DC3	U+0014 DC4	U+0015 NAK	U+0016 SYN	U+0017 ETB	U+0018 CAN	U+0019 EM	U+001A SUB	U+001B ESC	U+001C FS	U+001D GS	U+001E RS	U+001F US
2_	U+0020 SP	U+0021 !	U+0022 "	U+0023 #	U+0024 \$	U+0025 %	U+0026 &	U+0027 '	U+0028 (	U+0029)	U+002A *	U+002B +	U+002C ,	U+002D -	U+002E .	U+002F /
3_	U+0030 0	U+0031 1	U+0032 2	U+0033 3	U+0034 4	U+0035 5	U+0036 6	U+0037 7	U+0038 8	U+0039 9	U+003A :	U+003B ;	U+003C <	U+003D =	U+003E >	U+003F ?
4_	U+0040 @	U+0041 A	U+0042 B	U+0043 C	U+0044 D	U+0045 E	U+0046 F	U+0047 G	U+0048 H	U+0049 I	U+004A J	U+004B K	U+004C L	U+004D M	U+004E N	U+004F O
5_	U+0050 P	U+0051 Q	U+0052 R	U+0053 S	U+0054 T	U+0055 U	U+0056 V	U+0057 W	U+0058 X	U+0059 Y	U+005A Z	U+005B [	U+005C \]	U+005D ^	U+005E _	U+005F
6_	U+0060 `	U+0061 a	U+0062 b	U+0063 c	U+0064 d	U+0065 e	U+0066 f	U+0067 g	U+0068 h	U+0069 i	U+006A j	U+006B k	U+006C l	U+006D m	U+006E n	U+006F o
7_	U+0070 p	U+0071 q	U+0072 r	U+0073 s	U+0074 t	U+0075 u	U+0076 v	U+0077 w	U+0078 x	U+0079 y	U+007A z	U+007B {	U+007C 	U+007D }	U+007E ~	U+007F DEL

유니코드

- 인코딩
 - Unicode는 문자에 번호를 붙인 표준 (16진수)
 - 하지만 컴퓨터는 데이터를 바이트로 저장
 - 인코딩 Encoding : 문자 번호를 바이트로 바꾸는 방식
 - 문자 → 사람 눈에 보이는 글자 (가)
 - 유니코드 → 문자에 붙은 주민등록번호 (U+AC00)
 - 인코딩 → 그 번호를 컴퓨터 파일에 저장하는 포장 방식 (ea b0 80)
- UTF-8 : 대표적인 인코딩 방식

유니코드

- 인코딩
 - 문자 : ㄱ
 - 유니코드 : U+3131 (16진수)
 - UTF-8 : E3 84 B1 (3바이트)
 - 2진수 :
 - E3 → 11100011 (E : 1110 , 3 : 0011)
 - 84 → 10000100
 - B1 → 10110001
 - 총 크기:
 - 3바이트 = 24비트

유니코드

- 인코딩
 - 문자 : ㄱ
 - 유니코드 : U+3131 (16진수)
 - UTF-8 : E3 84 B1 (3바이트)
 - 2진수 :
 - E3 → 11100011 (E : 1110 , 3 : 0011)
 - 84 → 10000100
 - B1 → 10110001
 - 총 크기:
 - 3바이트 = 24비트

```
text = "ㄱ"
```

```
# 유니코드 번호를 16진수로 확인  
print(hex(ord(text))) # 0x3131
```

```
# 문자열 "ㄱ"을 UTF-8 방식으로 인코딩해서 bytes 타입으로 변환  
encoded_data = text.encode("utf-8")  
print(encoded_data) # b'\xe3\x84\xb1'
```

```
# 각 바이트를 2진수로 변환해서 출력  
print(" ".join(bin(byte)[2:].zfill(8) for byte in encoded_data))  
# 11100011 10000100 10110001
```

3-2 복사

복사

- 어떤 결과가 나올까?

```
numbers1 = [10, 20, 30]
numbers2 = numbers1

numbers2.append(40)

print(numbers1)
print(numbers2)
```

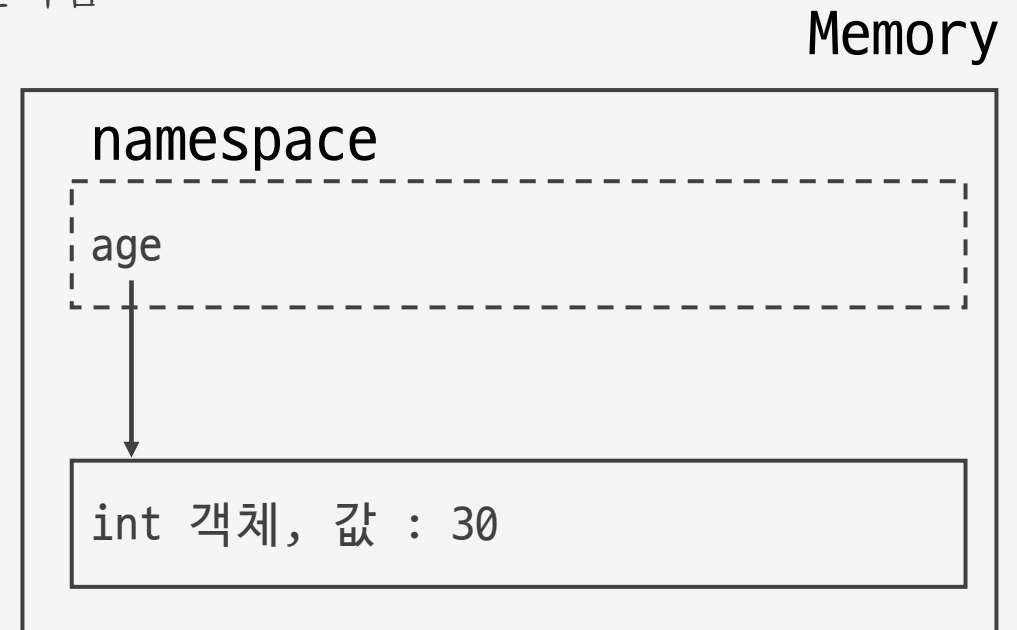
👉 왜 둘 다 바뀔까?

(복사) 참고 자료

- 변수와 메모리 할당
 - 변수는 값 자체가 아니라, 값을 가리키는 이름이다!
 - `id()` : 변수가 가리키는 객체의 주소 값을 반환하는 파이썬 내장 함수
 - 메모리 주소 `4387436232`에 30이라는 int 객체가 올라감
 - `age`라는 변수명은 namespace에 등록,
그 값을 가리키게 함

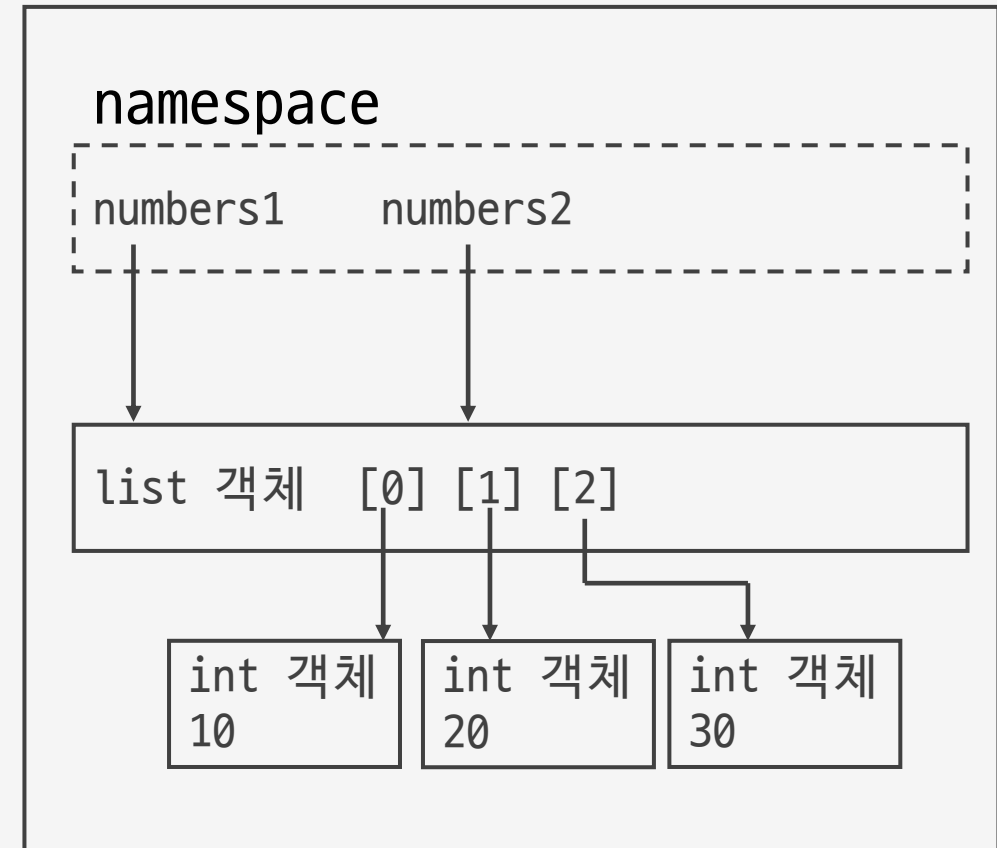
```
age = 30
```

```
print(id(age)) # 4387436232
```



복사

```
numbers1 = [10, 20, 30]  
numbers2 = numbers1
```



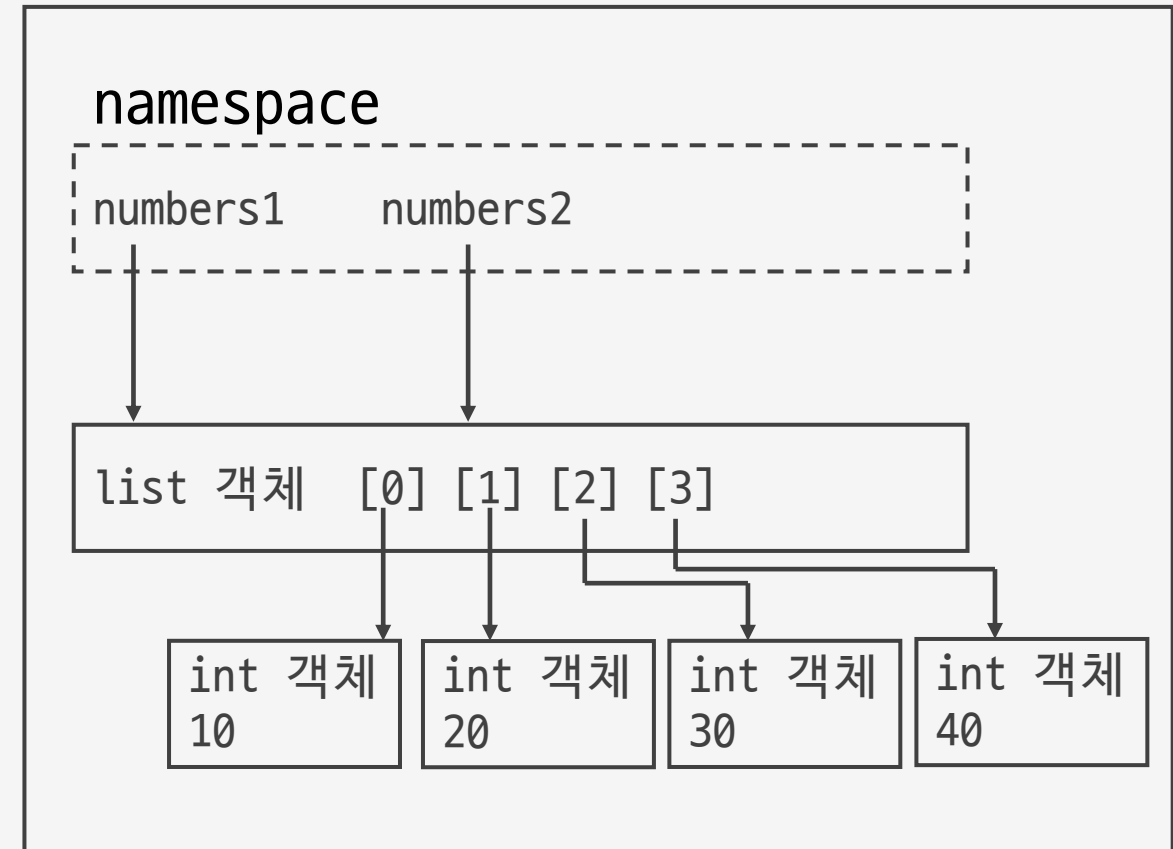
복사

- 어떤 결과가 나올까?

```
numbers2.append(40)
```

```
print(numbers1)
```

```
print(numbers2)
```



같은 리스트를 가리키는 이름표 하나를 더 만든 것 뿐!

복사

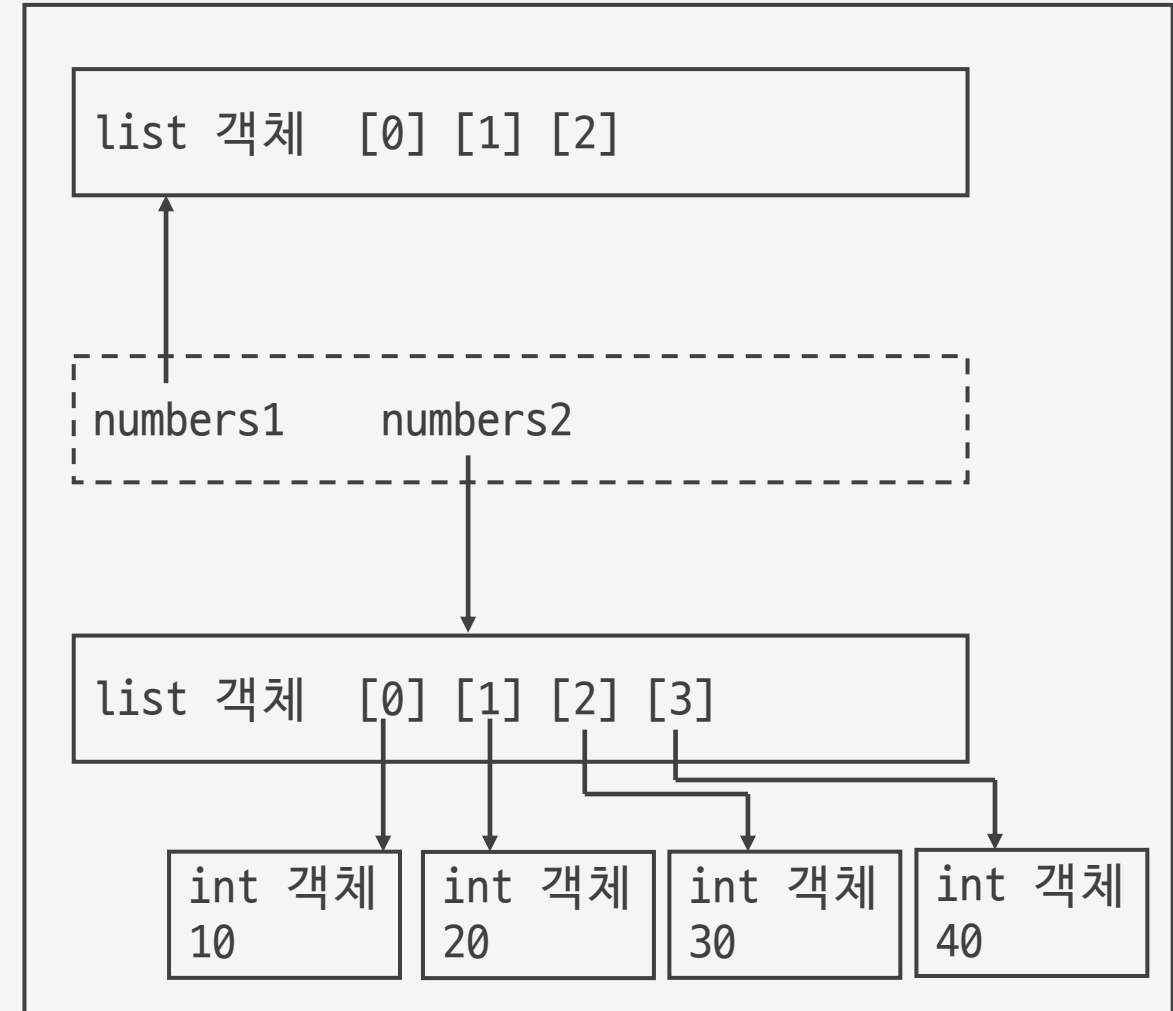
- 얕은 복사 (shallow copy)
 - 바깥쪽 컨테이너만 새로 복사하는 방식

1. `.copy()` 사용

```
numbers1 = [10, 20, 30]
numbers2 = numbers1.copy()

numbers2.append(40)

print(numbers1) # [10, 20, 30]
print(numbers2) # [10, 20, 30, 40]
```



복사

- 얕은 복사 (shallow copy)
 - 바깥쪽 컨테이너만 새로 복사하는 방식

2. : 사용

```
numbers1 = [10, 20, 30]
numbers2 = numbers1[:] # numbers1의 모든 요소를 복사하여 numbers2에 할당

numbers2.append(40)

print(numbers1) # [10, 20, 30]
print(numbers2) # [10, 20, 30, 40]
```

복사

- 얕은 복사 (shallow copy)
 - 바깥쪽 컨테이너만 새로 복사하는 방식

3. list() 사용

```
numbers1 = [10, 20, 30]
numbers2 = list(numbers1) # numbers1의 복사본을 생성

numbers2.append(40)

print(numbers1) # [10, 20, 30]
print(numbers2) # [10, 20, 30, 40]
```

복사

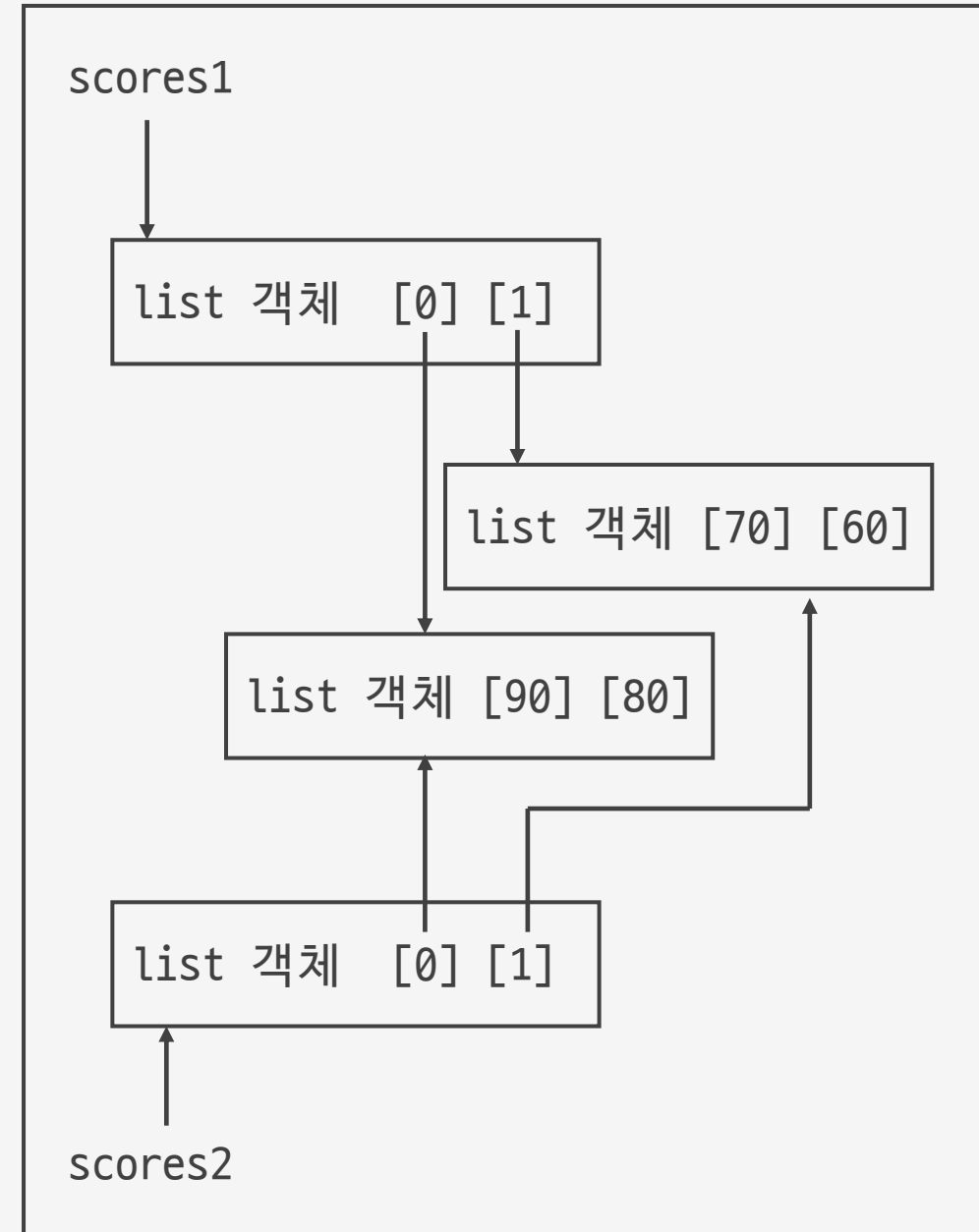
- 어떤 결과가 나올까?

```
scores1 = [[90, 80], [70, 60]]  
scores2 = scores1.copy()  
  
scores2[0].append(100)  
  
print(scores1)  
print(scores2)
```

복사

- 어떤 결과가 나올까?

```
scores1 = [[90, 80], [70, 60]]  
scores2 = scores1.copy()  
  
scores2[0].append(100)  
  
print(scores1)  
print(scores2)
```



복사

- 깊은 복사 (deep copy)
 - 바깥쪽 컨테이너 뿐만 아니라, 안쪽에 있는 객체까지 새로 복사하는 방식
 - copy라는 모듈 사용

```
import copy

scores1 = [[90, 80], [70, 60]]
scores2 = copy.deepcopy(scores1)

scores2[0].append(100)

print(scores1) # [[90, 80], [70, 60]]
print(scores2) # [[90, 80, 100], [70, 60]]
```


복사

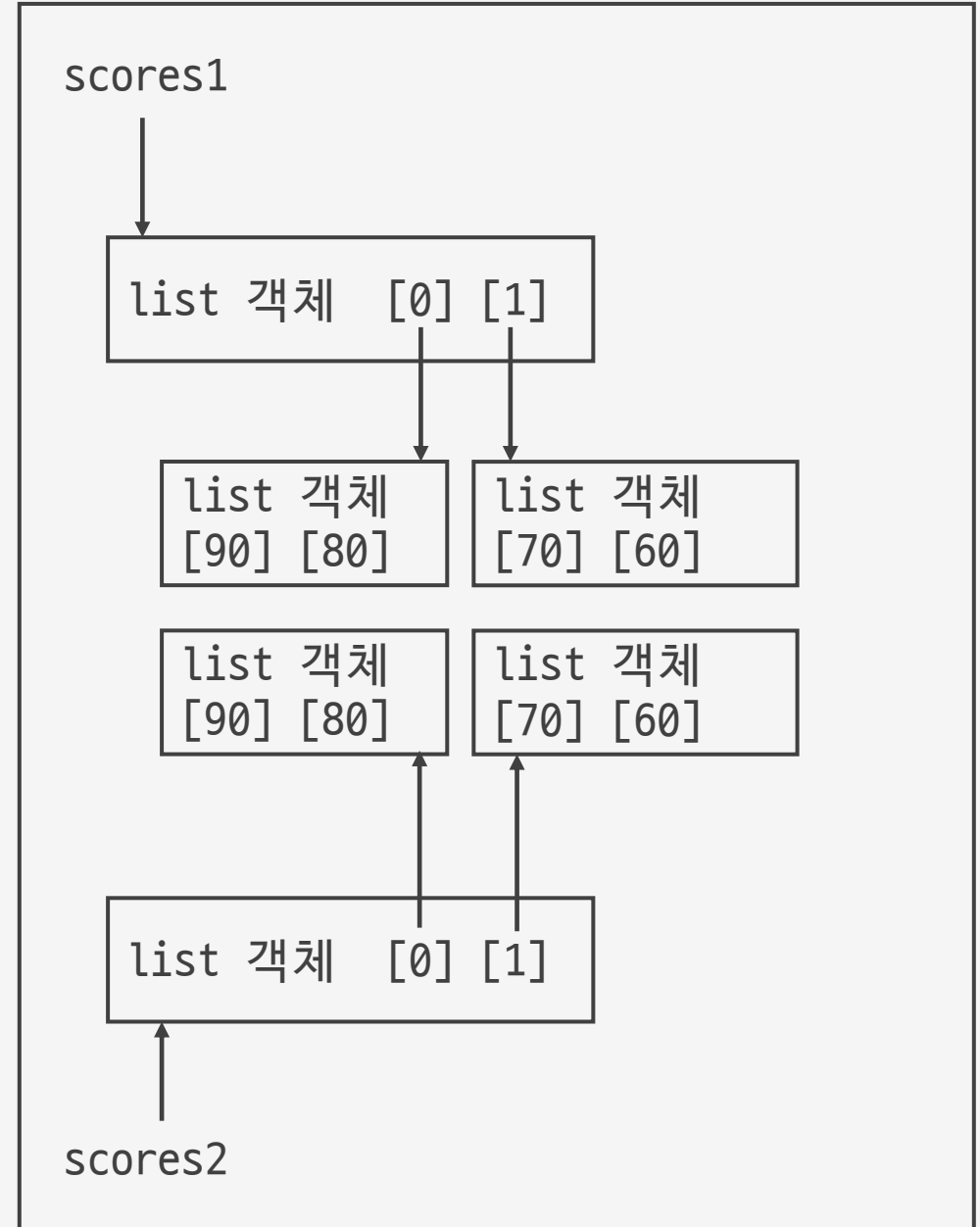
- 깊은 복사 (deep copy)

```
import copy

scores1 = [[90, 80], [70, 60]]
scores2 = copy.deepcopy(scores1)

scores2[0].append(100)

print(scores1) # [[90, 80], [70, 60]]
print(scores2) # [[90, 80, 100], [70, 60]]
```



복사

- 얇은 복사, 깊은 복사
 - 컨테이너 모두 적용

```
user1 = {  
    "name": "Jay",  
    "scores": [90, 80, 70]  
}  
  
user2 = user1.copy()  
  
user2["language"] = "Python"  
  
print(user1)  
print(user2)
```

```
import copy  
  
data1 = {  
    "user": "Jay",  
    "scores": [90, 80, 70],  
}  
  
data2 = copy.deepcopy(data1)  
  
data2["scores"].append(100)  
  
print(data1)  
print(data2)
```

수고하셨습니다.

